IsDB-BISEW

IT SCHOLARSHIP PROGRAMME

COURSE PROJECT PROPOSAL

BLOOM BASKET

Online Flower Shop E-Commerce System

A compact, class-based e-commerce application engineered for browsing flowers, managing catalog products and multi-status orders, and testing simulated checkout payment flows.

TRAINEE SUBMISSION

Submitted by:	Md Sumon
Trainee ID:	1297460
Batch:	PWAD/USSL-M/73/01
Course:	Web Development

PROJECT SUPERVISION

Project Type:	Core PHP & MySQL E-Commerce
TSP:	US SOFTWARE Limited
Instructor:	Syed Ziaul Habib
Consultant / Reviewer:	Md Moshaidul Islam

1. PROJECT OVERVIEW

Bloom Basket is a small online flower shop where customers can browse flower products, search and filter by category, add products to a shopping cart, place orders, and choose a payment method. Administrators manage categories, products, orders, and order statuses.

Project Objectives

- Build a functional e-commerce workflow using Core PHP and MySQL.
- Apply object-oriented programming, namespaces, Composer autoloading, and a custom routing system.
- Organize the application using a simple MVC architecture.
- Design a relational database with primary keys, foreign keys, and clear table relationships.
- Demonstrate checkout and simulated payment processing without handling real money.

Technology Stack

Technology	Use in the Project
Core PHP 8.3+ (OOP)	Application logic, classes, namespaces, controllers and services
MySQL 8.4 + PDO	Relational data storage and prepared database queries
Composer (PSR-4)	Class autoloading
Custom Router + MVC	URL routing and separation of models, views and controllers
Bootstrap 5 + JavaScript	Responsive interface and interactive frontend behavior

Project Scope

The first version will use a small catalogue of approximately 8–12 flower products, one administrator account, customer accounts, session-based carts, Cash on Delivery, and simulated bKash, Nagad, and Rocket payment options.

ISDB-BISEW IT SCHOLARSHIP PROGRAMME PWAD / USSL-M / 73 / 01

2. PROJECT FEATURES

Customer Features

Feature	Description
Account Management	Customer registration, login, and logout.
Browse Products	View the flower catalogue and individual product details.
Search & Categories	Search products by name and filter by flower category.
Shopping Cart	Add products, change quantities, and remove items. Cart data is stored in PHP sessions.
Checkout	Provide recipient name, contact number, and delivery address.
Payment Options	Choose Cash on Delivery or a simulated bKash, Nagad, or Rocket payment.
Order History	View previous orders, order details, and current order status.

Administrator Features

Feature	Description
Dashboard	View basic totals such as product count and order count.
Category Management	Create, view, update, and delete flower categories.
Product Management	Create, view, update, and delete products, including image and stock information.
Order Management	View orders and their items, customer details, and payment information.
Order Status	Update an order through pending, confirmed, processing, shipped, delivered, or cancelled.
Payment Records	Review payment method, amount, transaction reference, and payment status.

> **Payment Test-Mode Note:**
> The bKash, Nagad, and Rocket flows are simulated for demonstration only. The application will not connect to real payment-provider APIs or collect PINs/OTPs. A test screen can produce a success or failure result and save that result in the payments table. Real payment integration would require approved merchant credentials and official provider APIs.
>
> Order status and payment status are separate: a successful payment does not automatically mean an order has been delivered. Cash on Delivery can remain unpaid until collection.

3. DATABASE DESIGN

The database contains six main tables. The design separates product categories, order headers, order line items, and payment attempts to keep the data organized and reduce duplication.

Table	Purpose
`users`	Stores administrator and customer accounts. The role field distinguishes admin and customer.
`categories`	Stores flower categories such as Rose, Sunflower, Lily, Tulip, and Mixed Flowers.
`products`	Stores product name, description, price, stock, image path, and category reference.
`orders`	Stores each order's customer, delivery details, total amount, order number, and order status.
`order_items`	Stores the products, quantities, and prices captured at the time an order is placed.
`payments`	Stores payment method, amount, transaction reference, payment status, and payment timestamp.

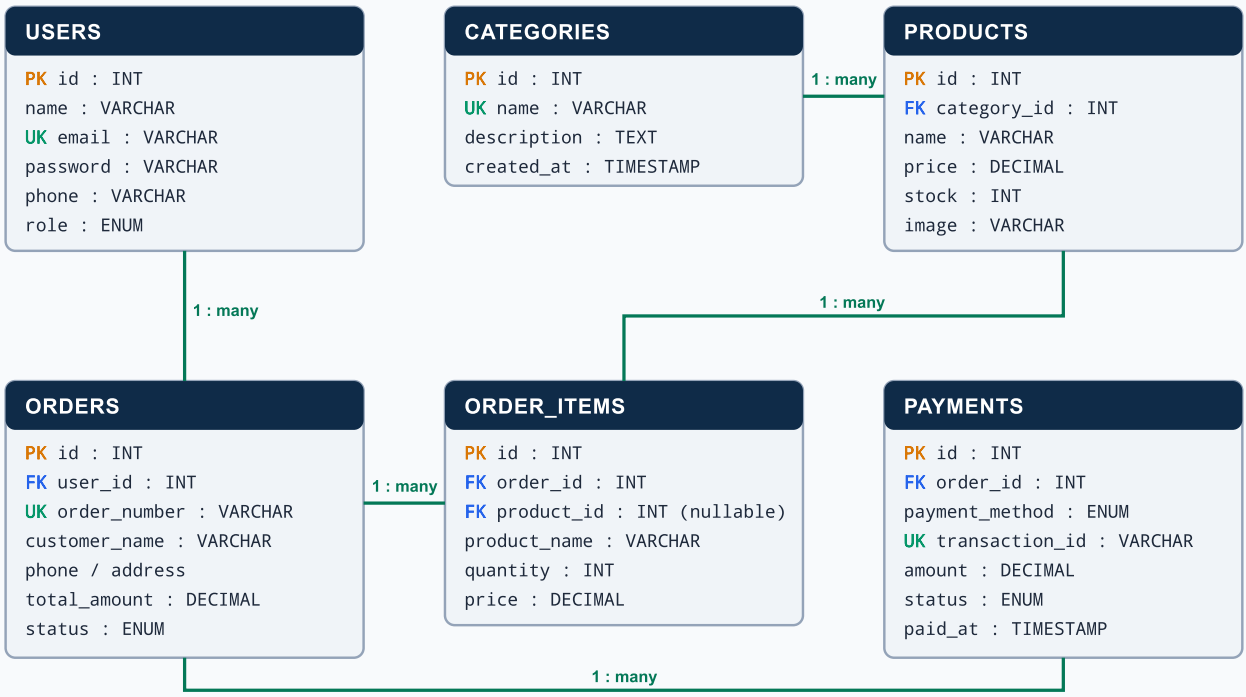
Important Database Decisions

- **Users and Orders:** One user can place many orders. Each order belongs to one user.
- **Categories and Products:** One category can contain many products. Each product belongs to one category.
- **Orders and Order Items:** One order contains one or more order-item rows.
- **Products and Order Items:** A product can appear in many order items. The product reference in an old order item may become `NULL` if the product is deleted; the saved product name and price remain.
- **Orders and Payments:** One order can have multiple payment attempts, for example a failed test payment followed by a successful attempt.

- **Historical Prices:** `order_items` stores `product_name` and `price` at purchase time so later product edits do not change old order records.

4. ENTITY RELATIONSHIP (ER) DIAGRAM

The diagram below displays the six core database entities and their structural key relationships. PK indicates Primary Key, FK indicates Foreign Key, and UK indicates Unique Key.



ISDB-BISEW IT SCHOLARSHIP PROGRAMME PWAD / USSL-M / 73 / 01

5. MAIN APPLICATION WORKFLOW

Customer Purchase Workflow

Step	Stage	Description
1	Browse	Customer opens the storefront, searches products, or filters by category.
2	Cart	Customer adds products to a PHP session cart and updates quantities.
3	Checkout	Server validates stock and calculates the total using prices from the database.
4	Order Creation	A database transaction creates the order and its `order_items`, then adjusts stock.
5	Payment	Customer selects COD or a simulated mobile payment method. Test payment results are recorded.
6	Order Tracking	Customer views order status; admin reviews the order and updates its status.

Application Architecture

Layer	Responsibility	Example
Routes	Map HTTP method and URL to a controller action.	`GET /products`
Controllers	Handle the request and coordinate application work.	`ProductController`
Models	Read and write relational data through PDO.	`Product`, `Order`
Services	Hold reusable business operations.	`CartService`, `PaymentService`
Views	Render HTML templates for customers and admins.	`products/index.php`
Middleware	Protect routes that require login or admin role.	`AuthMiddleware`

Planned Development Phases

1. Project setup, Composer autoloading, namespaces and configuration.
2. PDO Database class, Router, Request, Response and base Controller.
3. MVC integration and authentication with middleware.
4. Category and product management.
5. Customer storefront, search, category filtering and session cart.
6. Checkout, order creation, stock handling and payment simulation.
7. Admin order management, testing and security review.

6. PROJECT SCOPE AND QUALITY CONSIDERATIONS

Planned Limits for the First Version

Area	Decision
Products	Approximately 8–12 sample flower products.
Categories	A small fixed set of flower categories, manageable by admin.
Cart	PHP session-based cart; no separate cart database table.
Payments	Cash on Delivery and simulated bKash, Nagad, and Rocket.
Shipping	One simple delivery charge or free delivery rule.
Excluded Features	No wishlist, reviews, coupons, multi-vendor support, or real payment gateway in the first version.

Security and Data Integrity

- Use PDO prepared statements for database queries.
- Store passwords using `password_hash()` and verify them using `password_verify()`.
- Validate and authorize all admin actions on the server.
- Use CSRF protection for state-changing forms and escape output to reduce cross-site scripting risks.
- Validate uploaded image type and size; generate safe filenames and store uploads outside executable code paths where practical.
- Calculate prices and totals on the server; never trust totals submitted by the browser.
- Use database transactions for order creation, stock updates, and payment-state changes.
- Do not collect real mobile-wallet PINs or OTPs in the simulated payment flow.

Expected Outcome:
Bloom Basket will demonstrate a complete small-scale e-commerce workflow and practical understanding of PHP OOP, namespaces, routing, MVC, PDO, relational database design, sessions, authentication, order processing, and payment-state handling.

PROJECT PRESENTED BY

Md Sumon | Trainee ID: **1297460** | Batch: **PWAD/USSL-M/73/01**